

Synthetic Image Generation

https://github.com/aiyshwary/Synthetic_waste_image_generation

Python

Computer Vision

OpenCV

Data Augmentation

OVERVIEW

An automated pipeline for generating high-quality synthetic training images for object detection, addressing class-imbalanced datasets by compositing annotated object crops onto varied backgrounds with randomized augmentations.

IMPACT

Built an end-to-end synthetic image generation pipeline that composites real object crops onto diverse backgrounds with randomized augmentations, producing annotated training data that reduces dependency on costly manual data collection.

TECHNICAL WALKTHROUGH

This project builds an automated pipeline to generate high-quality synthetic images for object detection. The goal is to address limited or imbalanced real-world data by synthesizing new training images using real object crops and annotations.

Why synthetic data?

Manual annotation is expensive and time-consuming. Synthetic image generation allows us to increase dataset size, improve class balance, and expose models to diverse object placements without collecting new data.

End-to-end pipeline explanation (step-by-step, interview friendly)

Data ingestion from S3

The pipeline starts by downloading both annotated and unannotated sorted images from an S3 bucket. Duplicate downloads are avoided to ensure efficiency.

i) Unannotated images → for background usage

ii) Annotated images → for object extraction

Annotation handling

For annotated data, CVAT annotations in Dataro JSON format are downloaded and combined into a single unified annotation file.

Two paths

i) Convert existing annotations to YOLO format

Or auto-annotate using a pre-trained object detection model when annotations are missing

This ensures consistent labeling across the dataset.

Dataset consolidation

All images and YOLO labels are flattened into unified folders to standardize downstream processing.

This step simplifies

i) Crop generation

ii) Mask creation

iii) Synthetic placement

Object crop & mask generation

From the labeled dataset, object-level crops and corresponding binary masks are generated.

Outputs

i) Object crops

ii) Pixel-level masks

iii) Crop-level labels

These form the building blocks for synthetic images.

Synthetic image generation

Using the extracted object crops and masks, new images are synthesized by placing objects on background images.

Two strategies

Spread-out placement → realistic non-overlapping scenes
Clustered placement (optional) → congested scenes with object repetition

This increases

i) Spatial diversity

ii) Object density variation

iii) Robustness of detection models

Phase-based synthetic generation

An optional phase-based generator creates images in multiple stages, gradually increasing object density and class balancing.

This helps

- i) Control dataset distribution
- ii) Target specific class counts
- iii) Improve long-tail performance

Visualization & validation

Finally, synthetic images are visualized with bounding boxes overlaid from YOLO labels to verify annotation quality.

This acts as a sanity check before training.

Tech stack & reproducibility

The entire pipeline is Dockerized, ensuring reproducibility across environments.

Dependencies include OpenCV, NumPy, Pandas, and AWS SDK (boto3) for seamless S3 integration.

One-line summary

I built a fully automated, Dockerized synthetic data generation pipeline that converts CVAT annotations into YOLO format, extracts object crops, and synthesizes diverse training images to improve object detection performance.

If interviewer asks "What impact does this have?"

You can say

This pipeline significantly reduces annotation cost, improves class balance, and enables scalable dataset generation, leading to more robust object detection models in data-constrained environments.

KEY DETAILS

1. Extracts foreground object crops from existing annotated datasets using bounding box and mask information.
2. Composites crops onto varied background images with randomized scale, rotation, and position transformations.
3. Automatically generates YOLO/Pascal VOC-compatible annotation files alongside each synthetic image.
4. Targets underrepresented classes to balance dataset distributions for improved model generalization.
5. Validated improvements in downstream detection model performance on augmented versus original-only training sets.

6. Uses Ultralytics `auto_annotate` with a custom YOLO detection model and SAM2 for automatic polygon-level instance segmentation labels.

7. Implements three synthesis strategies: `CropScatterSynth` (up to 30 objects/image), `ClusteredSyntheticGen` (40,000 instances/class target), and `SyntheticPhaseGen` (three-phase YAML-configurable generation) with multithreaded generation.

8. Dockerized on `nvidia/cuda:12.2.0` with full 9-step bash pipeline: S3 download, annotation conversion, auto-annotation, crop extraction, synthetic generation, and visualization.